Lecture - 5 Java Type Conversion & Type Promotions

⇒ type conversion → converting one dataType to another.

Types:

1.) Implicit → done by Java

2.) Explicit → done by us

```
byte b = 42;
int i;
i = b;
sout(i) → 42   how
```

⇒ Rules for implicit conversion

→ destination data type should be wider than Source datatype

int i = b → (1 byte)

↙ wider datatype (4 byte)

→ so here byte is smaller than int.

→ This is automatically done by Java. and this is called widening Conversion.

→ short → int] possible
int → long]

⇒ Explicit Conversion

→ now here destination datatype will be smaller than source. so Java will not convert it, we will have to do this manually.

```
→ int i = 300;
  byte b;
  b = i; → compiler error (cannot convert int to byte)
```

because according to Java, it will lead to data loss)

⇒ b = (byte) i; → valid

→ now it will truncate data   300 → 100101100

→ but byte can hold max 8 bits so it will keep only 8 bits from LSB

$\Rightarrow 2^2 + 2^3 + 2^5$

$\Rightarrow 4 + 8 + 32$

$\Rightarrow 44$

So this can leads to data loss.

simpler formula ⇒ value % (Range of same data type)

⇒ 300 % 256 ⇒ 44

long → int
double → float
> Narrowing Conversion

⇒ char c = 'a';
int a = c ; sout (a); → 97

⇒ Truncating Conversion

Float
Double
> Int

Float f = 16.25f;
int i ;
i = f; → error (no implicit type casting);

i = (int) f; →
Sout (i); → 16 (only decimal part)

Same with double.

* Implicit Valid conversions

byte → short, int, long, float, double
short → int, long, float, double
char → int, long, float, double
int → long, float, double
long → float, double
float → double

→ Boolean type conversion to any other data type is not possible?

* Type Promotion in Java

byte a = 50;
byte b = 40;
byte c = 100;
int i = (a×b)/c;

→ now range of byte → -128 to 127 and since if these variables can have max value so how Java will store so it will convert convert a, b, c to int to perform this calculation. So that it can store result in a valid range

$\boxed{\text{byte} \rightarrow \text{int;}}$

some too byte b = 50;

b = b * 2; → error as b will be type promoted to int

int c = b×2; → valid or
byte c = (byte) b×2;

* Type Promotion rules

1.) byte, short, char values are promoted to int.
2.) If one operand is long, then whole expression will become long
3.) If one operand is float, entire expression will become float.
4.) If one operand is double, entire expression is double.

eg→
```
byte b = 42;
char c = '0';
short s = 1024;
int i = 50000;
float f = 5.67f;
double d = .1234;
```

double result = $\underbrace{(f * b)}_{\text{float}} + \underbrace{(i / c)}_{\text{int}} - \underbrace{(d * s)}_{\text{double}}$;

→ now highest storage capacity is of double so end result will be double.

as float + int → float
float * double → double